TEMPLATE METHOD APLICADO EN OPERACIONES MATEMÁTICAS BÁSICAS

Patrones de Diseño: *Template Method*

Definición del esqueleto de un algoritmo de cálculo desacoplado de la interfaz de usuario

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Octubre 2026

Índice

1. Diagrama UML **2**

2. Explicación de las Clases **3**
2.1. Clase Abstracta (Template): Operacion 3
2.2. Subclases Concretas (Operaciones) 3
2.3. Interfaz de Vista y sus Implementaciones 5

3. Pruebas **6**
3.1. Clase Cliente 6

4. Repositorio **8**
4.1. Enlace GitHub 8

Diagrama UML

Descripción General

El proyecto implementa el patrón de diseño **Template Method**, un patrón comportamental que permite definir la estructura esqueleto de un algoritmo en una superclase abstracta, permitiendo que las subclases sobrescriban pasos específicos del algoritmo (como la operación o la validación) sin cambiar la estructura general del mismo. Además, desacopla la entrada y salida de datos utilizando el patrón de interfaz de Vista.

Diagrama de Clases

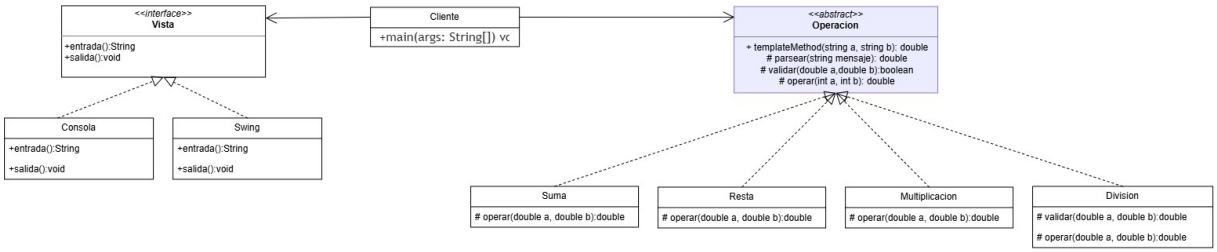


Figura 1: Diagrama de Clases del Sistema (Patrón Template Method).

Explicación de las Clases

Clase Abstracta (Template): Operacion

Descripción

`Operacion` es la clase abstracta que define el método plantilla `templateMethod`. Contiene los pasos de parsear datos, validar restricciones de negocio y delegar la operación aritmética concreta a sus subclases.

Código Fuente: Operacion.java

```java
package com.mycompany.operaciones.templatemethod;

public abstract class Operacion {

    public final double templateMethod(String textoA, String
       textoB) {
        double a = parsear(textoA);
        double b = parsear(textoB);
        if (validar(a, b)) {
            return operar(a, b);
        }
        throw new IllegalArgumentException("Error al validar los
           n meros");
    }

    public final double parsear(String mensaje) {
        try {
            return Double.parseDouble(mensaje);
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Error al parsear
               el n mero");
        }
    }

    public abstract double operar(double a, double b);

    public boolean validar(double a, double b) {
        return true;
    }
}
```

Subclases Concretas (Operaciones)

Descripción

Clases que extienden `Operacion` e implementan o personalizan únicamente los métodos requeridos (`operar` y opcionalmente `validar`).

Código Fuente: Suma.java

```java
package com.mycompany.operaciones.templatemethod;

public class Suma extends Operacion {
```

```
4       @Override
5       public double operar(double a, double b) {
6           return a + b;
7       }
8   }
```

Código Fuente: Division.java

```
1   package com.mycompany.operaciones.templatemethod;
2
3   public class Division extends Operacion {
4       @Override
5       public boolean validar(double a, double b) {
6           return b != 0;
7       }
8
9       @Override
10      public double operar(double a, double b) {
11          return a / b;
12      }
13  }
```

Interfaz de Vista y sus Implementaciones

Descripción

La interfaz `Vista` y sus clases concretas (`Consola` y `Swing`) independizan la interacción de usuario de la lógica matemática.

Código Fuente: Vista.java

```
1 package com.mycompany.operaciones.templatemethod;
2
3 public interface Vista {
4     String entrada();
5     void salida(String mensaje);
6 }
```

Pruebas

Clase Cliente

Descripción

La clase `Cliente` actúa como el orquestador del sistema. Permite elegir la interfaz de usuario deseada y ejecuta las distintas operaciones matemáticas enviando los datos obtenidos.

Código Fuente: Cliente.java

```
1  public class Cliente {
2
3      public static void main(String[] args) {
4
5          String opcion = JOptionPane.showInputDialog("Ingrese␣la␣
               opcion␣de␣la␣vista:␣\n␣1.␣Consola␣\n␣2.␣Swing");
6
7          while(!opcion .equals ("1") && !opcion.equals("2")){
8                      JOptionPane.showMessageDialog(null, "Opcion␣
                           invalida");
9                      opcion = JOptionPane.showInputDialog("Ingrese
                           ␣la␣opcion␣de␣la␣vista:␣\n␣1.␣Consola␣\n␣2.
                           ␣Swing");
10          }
11
12         Vista vista = null;
13
14         if(opcion.equals("1")){
15                     vista = new Consola();
16         }else if (opcion.equals("2")){
17                     vista = new Swing();
18         }
19
20         String a = vista.entrada();
21         String b = vista.entrada();
22
23         Operacion opSuma = new Suma();
24         vista.salida("Resultado␣de␣"+a+"␣+␣"+b);
25         try{
26         double suma = opSuma.templateMethod(a, b);
27         vista.salida(String.valueOf(suma));
28         }catch(IllegalArgumentException e){
29         vista.salida("Error:␣" + e.getMessage());
30         }
31
32         Operacion opResta = new Resta();
33         vista.salida("Resultado␣de␣"+a+"␣-␣"+b);
34         try{
35         double resta = opResta.templateMethod(a, b);
36         vista.salida(String.valueOf(resta));
37         }catch(IllegalArgumentException e){
```

```
38              vista.salida("Error: "+e.getMessage());
39          }
40
41          Operacion opMultiplicacion = new Multiplicacion();
42          vista.salida("Resultado de "+a+" * "+b);
43          try{
44          double multiplicacion = opMultiplicacion.templateMethod(a
                , b);
45          vista.salida(String.valueOf(multiplicacion));
46          }catch(IllegalArgumentException e){
47              vista.salida("Error: "+e.getMessage());
48          }
49
50          Operacion opDivision = new Division();
51          vista.salida("Resultado de "+a+" / "+b);
52          try{
53          double division = opDivision.templateMethod(a, b);
54          vista.salida(String.valueOf(division));
55          }catch(IllegalArgumentException e){
56              vista.salida("Error: "+e.getMessage());
57          }
58      }
59  }
```

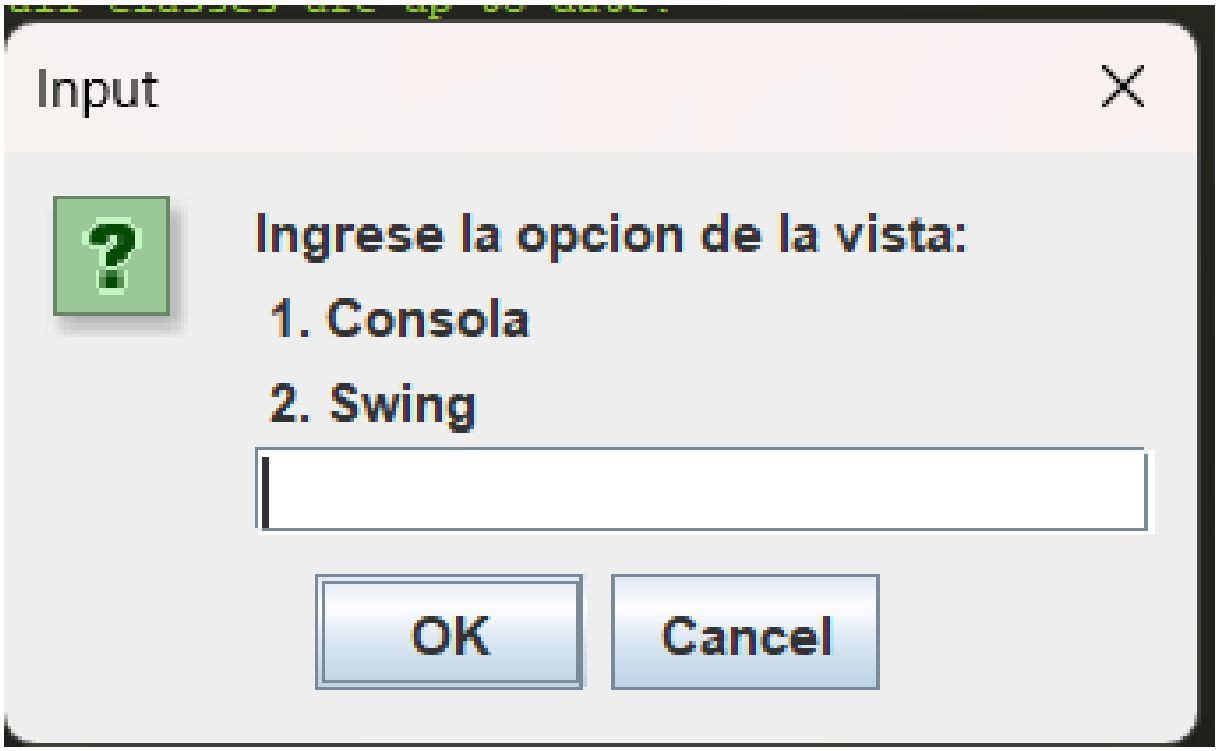


Figura 2: Salida Consola.

```
--- resources:3.3.1:resources (default-resources) @ Operaciones-TemplateMethod ---
skip non existing resourceDirectory C:\Nath\UD\Modelos de Programacion\Patrones\Patrones Comportamiento

--- compiler:3.13.0:compile (default-compile) @ Operaciones-TemplateMethod ---
Nothing to compile - all classes are up to date.

--- exec:3.5.1:exec (default-cli) @ Operaciones-TemplateMethod ---
Entrada:
5
Entrada:
3
Resultado de 5 + 3
8.0
Resultado de 5 - 3
2.0
Resultado de 5 * 3
15.0
Resultado de 5 / 3
1.6666666666666667
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  05:08 min
Finished at: 2026-10-06T21:02:00-05:00
------------------------------------------------------------------------
```

Figura 3: Salida Consola 2.

Repositorio

Enlace GitHub

- https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Comportamiento/Operaciones-TemplateMethod